axiateam
RDT company

Módulo 9

Genéricos

Curso: TypeScript

90 min

Objetivos de Aprendizaje

- Comprender el concepto de **genéricos**
- Crear **funciones genéricas** reutilizables
- Implementar **clases e interfaces** genéricas
- Aplicar **restricciones** a genéricos
- Utilizar genéricos **built-in** de TypeScript

1. Contexto

Reutilización de Código

El Problema

Sin genéricos, necesitamos repetir código:

```
// Versión para strings
function identityString(value: string): string {
  return value;
}

// Versión para números
function identityNumber(value: number): number {
  return value;
}

// Versión para booleanos
function identityBoolean(value: boolean): boolean {
  return value;
}
```

La Solución: Genéricos

```
function identity<T>(value: T): T {
  return value;
}

identity<string>("hello");
identity<number>(42);
identity(true); // TypeScript infiere el tipo
```

2. Funciones Genéricas

Generic Functions

```
// Función genérica básica
function firstElement<T>(arr: T[]): T | undefined {
  return arr[0];
}

// Múltiples tipos genéricos
function pair<K, V>(key: K, value: V): [K, V] {
  return [key, value];
}

// Restricciones con extends
function getProperty<T, K extends keyof T>(obj: T, key: K): T[K] {
  return obj[key];
}
```

Lab: Utilidades Genéricas para Tareas

Objetivo: Crear funciones genéricas reutilizables.

1. Implementa `wrapInArray`
2. Implementa `mapArray`
3. Implementa `filterByProperty`
4. Implementa `sortByProperty`

Código del Lab

```
function wrapInArray<T>(value: T): T[] {
  return [value];
}

function mapArray<T, U>(arr: T[], fn: (item: T) => U): U[] {
  return arr.map(fn);
}

function filterByProperty<T, K extends keyof T>(
  arr: T[],
  key: K,
  value: T[K]
): T[] {
  return arr.filter(item => item[key] === value);
}

function sortByProperty<T, K extends keyof T>(
  arr: T[],
  key: K
```

3. Clases Genéricas

Generic Classes

```
// Repository genérico
class Repository<T> {
  private items: T[] = [];

  add(item: T): void {
    this.items.push(item);
  }

  getAll(): T[] {
    return [...this.items];
  }

  find(predicate: (item: T) => boolean): T[] {
    return this.items.filter(predicate);
  }
}

const taskRepo = new Repository<Task>();
const userRepo = new Repository<User>();
```

Lab: Repository Genérico para Tareas

Objetivo: Implementar un repositorio genérico.

1. Crea la clase `TaskRepository`
2. Implementa métodos CRUD
3. Añade métodos de búsqueda

4. Restricciones (Constraints)

Constraints con extends

Las restricciones limitan los tipos permitidos:

```
// Requiere que T tenga id
interface Identifiable {
  id: string;
}

function findById<T extends Identifiable>(
  items: T[],
  id: string
): T | undefined {
  return items.find(item => item.id === id);
}

// Múltiples restricciones
function createPair<K extends string, V extends { toString(): string }>(
  key: K,
  value: V
): { key: K; value: string } {
  return { key, value: value.toString() };
}
```

Lab: Sistema de Filtrado Genérico

Objetivo: Crear filtros genéricos con restricciones.

1. Define interfaz base `Filterable`
2. Implementa filtro genérico con `extends`
3. Añade múltiples criterios de filtrado

```
interface Filterable {
  status?: TaskStatus;
  priority?: TaskPriority;
}

class TaskFilter<T extends Filterable> {
  filter(items: T[], criteria: Partial<T>): T[] {
    return items.filter(item => {
      for (const key in criteria) {
        if (criteria[key] !== undefined &&
            item[key] !== criteria[key]) {
          return false;
        }
      }
      return true;
    });
  }
}
```

Resumen

Puntos Clave

- Los **genéricos** permiten crear código reutilizable con tipos variables
- `<T>` define un parámetro de tipo
- **Restricciones** con `extends` limitan los tipos permitidos
- TypeScript infiere el tipo en la mayoría de casos
- **Utility types** como `Partial`, `Required`, `Pick`

Ejercicio Integrador

Sistema de Biblioteca

Caso 2: Biblioteca

Objetivo: Aplicar genéricos al sistema de biblioteca.

1. Crea un repositorio genérico para libros y miembros
2. Implementa filtros genéricos
3. Crea utility types para el dominio

Trabajo en grupos de 2-4 personas.

Tips y Gotchas

✅ Tips

- Genéricos permiten código reusable manteniendo tipos
- Usa `extends` para constraints
- Utility types: `Partial<T>`, `Pick<T,K>`, `Omit<T,K>`

⚠️ Gotchas

- `Array<T>` vs `T[]` son equivalentes
- Constraint `extends` no es herencia
- No abuses de `any` genérico - especifica constraints

Referencias

Recursos Adicionales

- Generics
- Utility Types

Gracias!

Módulo 9 Completado

Siguiente: Módulo 10 - Módulos